

Class: 17

08/05/2025

Types of Variables

→ 3 types of variables.

1. Instance Variables — ~~code~~ which you declare outside all methods & inside a class.
2. Static Variables → stores inside heap memory.
3. Local Variables.

Instance Variable:

Ex: Public class Animal {

```
    void public mt ( ) String name; → Instance Variable.  
    Static Variable Static int age; [which are accessible  
    PS VM { from anywhere]  
    }  
  
    void main ( ) {  
    } → Instance method.
```

- JVM gives default values to instance variables.
- We need an object to access instance variable.
- Also called as Global variables in other languages.

* Instance methods:

- We can access in entire class.

Static Variables:

→ It ~~stores~~ declare outside all methods & inside a class.

→ Stores in method area.

→ It loaded when a class is loaded.

[Class is loaded by CLSS]

→ These don't require an object.

→ If we create a static variable in class, memory will assign at that time only.

Ex: public class Animal {

String name; → Instance Variables.

Static int age; → Static Variable.

(JVM access this with creating an object)

(to access without object) ←

public static void main (String[] args) {

(* JVM cannot create object)

int a = 10; → Local Variable

}

void m1() {

int b = 20; → Local Variable.

}

}

Local Variables:

- It declares inside any method.
- Stores in Stack area.
- When we call the method, memory will ~~also~~ be assigned to the local variable (i.e., inside the method()).
- Stack works for temporary purpose, we can't access the variable without calling that method.
- Stack stores only when the code is in execution process, after the completion of execution the data will be erased.
- Due to this, local variables have ~~blue~~ **block scope**.
- We can ~~not~~ access that local variable inside that method only.
- We don't need an object to access local variables.
- JVM will not give any default values to local variables.

* **Static Variables** are used,

- When your value is acting as a constant.
- It allocates memory only once, if we use multiple times.
- we can change the value of static variables, but we never change due to all objects share that data, when we update

Ex: package com;

```
public class Company {  
    Static String companyName; (all objects share common data)  
    // [we can access directly with class name]
```

```
    String location;
```

```
    int numberOfEmployees;
```

```
    public Company() { }  
    public Company(String companyName, String location, int numberOfEmployees) {
```

```
        companyName = companyName
```

```
        location = location
```

```
        numberOfEmployees = numberOfEmployees.  
    }
```

package com;

public class test;

psvm {

```
    Company myCompany = new Company();
```

```
    Company. Company. companyName = "FLM";
```

```
    // myCompany. companyName = "xyz";
```

```
    myCompany. location = "Hyd";
```

```
    myCompany. numberOfEmployees = 12000;
```

```
    Company friendCompany = new Company();
```

```
    // friendCompany. Company. companyName = "xyz";
```

```
    friendCompany. location = "Bangalore";
```

```
    friendCompany. numberOfEmployees = 25000;
```

```
    Company sys0 (myCompany. companyName);
```

```
    sys0 (myCompany. location);
```

```
    sys0 (myCompany. numberOfEmployees);
```

```
    sys0 (friendCompany. companyName);
```

```
    sys0 (friendCompany. location);
```

Output: xyz
Hyd
12000

xyz
Bangalore
25000